**SENA — Servicio Nacional de Aprendizaje**

Tecnólogo en Análisis y Desarrollo de Software

Ficha de formación: 3145556

Centro de formación: Neiva, Huila

# Informe de Pruebas Unitarias

Guardián Escolar

Integrantes del proyecto:

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Neiva, 6 de octubre de 2026

# Tabla de contenido

**Información General** ..... **0**
**Resumen Ejecutivo** ..... **0**
Servicios Evaluados ..... 0
**Plan de Pruebas** ..... **0**
Objetivo ..... 0
Alcance ..... 0
Estrategia ..... 0
Ambiente y datos ..... 0
Criterios de entrada y salida ..... 0
Roles y responsabilidades ..... 0
Evidencia ..... 0
**Stack Tecnológico por Servicio** ..... **0**
**Detalle por Servicio** ..... **0**
1. ms-fleet (Gestión de Flota) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0
2. ms-iam (Identidad y Acceso) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Maven) ..... 0
3. ms-notification (Notificaciones e Incidentes) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0
4. ms-route (Rutas y Viajes) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0
5. ms-school-management (Gestión de Escuelas y Sedes) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0
6. ms-user-management (Gestión de Usuarios y Familias) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0
7. sg-ms-forgot-information (Recuperación de Información) ..... 0
Casos de Uso / Historias de Usuario Cubiertas ..... 0
Código de Pruebas Representativas ..... 0
Evidencia de Ejecución (Docker Compose) ..... 0

**Arquitectura de Pruebas y Patrones Utilizados** ..... **0**
Patrones Comunes en Todos los Servicios ..... 0
**Cobertura de Requerimientos Funcionales (FR)** ..... **0**
**Métricas de Calidad** ..... **0**
Resumen Global ..... 0
Por Servicio ..... 0
**Conclusiones** ..... **0**
✅ Fortalezas ..... 0
📋 Recomendaciones ..... 0
**Anexos** ..... **0**
A. Comandos de Ejecución por Servicio ..... 0
B. Requisitos Previos ..... 0
C. Archivos de Configuración de Tests ..... 0

## Información General

| Campo | Detalle |
|---|---|
| **Título** | Informe de Pruebas Unitarias |
| **Integrantes** | Sharik Dayanna Rojas Ibarra, Johan Smith Santamaria Fernández y Juan Pablo Chala Ramírez |
| **Ficha** | 3145556 |
| **Programa** | Tecnólogo en Análisis y Desarrollo de Software |
| **Fecha** | 25 de octubre de 2026 |
| **Proyecto** | School Guardian - Backend |

---

## Resumen Ejecutivo

Este documento presenta el informe detallado de las pruebas unitarias ejecutadas para cada uno de los microservicios del backend de School Guardian (excluyendo el servicio GPS - `sg-ms-gps` según lo solicitado). Todas las pruebas se ejecutaron mediante Docker Compose utilizando los perfiles de test configurados en cada servicio, garantizando un entorno aislado y reproducible.

### Servicios Evaluados

| Servicio | Tecnología | Tests Ejecutados | Tests Pasados | Tests Fallados | Estado |
|---|---|---|---|---|---|
| ms-fleet | .NET 10 / C# / NUnit | 39 | 39 | 0 | ✅ PASS |
| ms-iam | Java 21 / Spring Boot / | 57 | 57 | 0 | ✅ PASS |

| Servicio | Tecnología | Tests Ejecutados | Tests Pasados | Tests Fallados | Estado |
|---|---|---|---|---|---|
| | JUnit 5 / Mockito | | | | |
| ms-notification | .NET 10 / C# / NUnit | 56 | 56 | 0 | ✅ PASS |
| ms-route | .NET 10 / C# / NUnit | 74 | 74 | 0 | ✅ PASS |
| ms-school-management | .NET 10 / C# / NUnit | 22 | 22 | 0 | ✅ PASS |
| ms-user-management | .NET 10 / C# / NUnit | 66 | 66 | 0 | ✅ PASS |
| sg-ms-forgot-information | .NET 10 / C# / NUnit | 62 | 62 | 0 | ✅ PASS |
| **TOTAL** | | **376** | **376** | **0** | ✅ **TODOS PASS** |

---

## Plan de Pruebas

### Objetivo

Verificar que la lógica de negocio de cada microservicio del backend cumple su contrato antes de la entrega: casos de uso, validaciones, reglas de dominio, mapeos y adaptadores. Las pruebas son unitarias y de integración de componente; no sustituyen las pruebas de extremo a extremo sobre el sistema desplegado.

### Alcance

| Dentro del alcance | Fuera del alcance |
|---|---|
| Los siete servicios con suite de pruebas: `ms-fleet`, `ms-iam`, `ms-notification`, `ms-route`, `ms-` | `sg-ms-gps`, excluido por decisión del equipo al no |

| Dentro del alcance | Fuera del alcance |
|---|---|
| `school-management`, `ms-user-management` y `sg-ms-forgot-information`. | contar con suite estable en la versión entregada. |
| Casos de uso, dominio, validaciones, mapeos y adaptadores cubiertos por cada suite. | Pruebas de carga, estrés, penetración y usabilidad; se registran como trabajo futuro. |
| Ejecución reproducible en contenedores con el perfil `tests`. | Disponibilidad o rendimiento del entorno desplegado en AWS. |

**Estrategia**

1. Cada servicio declara su suite en un proyecto de pruebas (`tests/`) y un perfil `tests` en su `docker-compose.yml`.
2. La ejecución se hace con `dotnet test` para los servicios .NET y `mvn test` para `ms-iam`, dentro de la imagen de SDK correspondiente.
3. Los dobles de prueba son repositorios en memoria y servicios falsos (`FakeIamService`, `FakeGpsDeviceService`, `FakeTenantProvider`), de modo que ninguna prueba depende de la base de datos ni de servicios externos.
4. Cada caso se asocia a un caso de uso o historia (`CU-*`) y se agrupa por clase de prueba.
5. El resultado se registra por servicio (tests ejecutados, aprobados, fallidos y tiempo) y se consolida en las métricas de calidad.

**Ambiente y datos**

| Elemento | Configuración |
|---|---|
| Runtime | .NET 10 para seis servicios; Java 21 con Spring Boot para `ms-iam` |
| Frameworks de prueba | NUnit 4.x en .NET; JUnit 5 con Mockito y AssertJ en Java |
| Contenedores | Docker Compose con perfil `tests` por servicio |
| Datos | En memoria o generados por la prueba; no se usan |

| Elemento | Configuración |
|---|---|
| | datos reales de usuarios |
| Configuración | Variables y archivos de prueba versionados; sin credenciales de producción |

## Criterios de entrada y salida

| Criterio | Definición |
|---|---|
| Entrada | El servicio compila y su proyecto de pruebas restaura dependencias. |
| Salida | La suite termina sin fallos y el resumen se registra en este informe. |
| Aceptación | 100 % de las pruebas ejecutadas en verde y cobertura de los casos de uso declarados. |
| Rechazo | Cualquier prueba fallida, omitida sin justificación o caso de uso sin cubrir. |

## Roles y responsabilidades

| Rol | Responsabilidad |
|---|---|
| Integrantes del equipo | Escribir y mantener las pruebas de los servicios asignados. |
| Responsable de calidad | Ejecutar la suite completa, consolidar métricas y reportar desviaciones. |
| Instructor | Revisar el informe y contrastar los resultados con el repositorio. |

## Evidencia

La evidencia de ejecución es el comando reproducible de cada servicio y su resultado registrado en la sección «Detalle por Servicio» (por ejemplo, `39 tests pasados en 313 ms`),

más las métricas consolidadas y los comandos del Anexo A. Los registros crudos de consola no se adjuntan como archivos; se pueden reproducir con los comandos indicados.

---

**Stack Tecnológico por Servicio**

| Servicio | Lenguaje / Runtime | Framework Principal | Testing Framework | Librerías de Testing Adicionales | Build / Ejecución Tests |
|---|---|---|---|---|---|
| **ms-fleet** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, FluentAssertions (implícito via NUnit constraints) | `dotnet test` via Docker Compose profile `tests` |
| **ms-iam** | Java 21 | Spring Boot 3.2+ | **JUnit 5 (Jupiter)** | Mockito 5.x, AssertJ, Spring Boot Test, Testcontainers (config) | `mvn test` via Maven Surefire/Failsafe |
| **ms-notification** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, EntityFrameworkCore.InMemory, AutoMapper | `dotnet test` via Docker Compose profile `tests` |
| **ms-route** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, AutoMapper | `dotnet test` via Docker Compose profile `tests` |
| **ms-school-management** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, AutoMapper, Microsoft.Extensions.Caching. | `dotnet test` via Docker Compose profile `tests` |

| Servicio | Lenguaje / Runtime | Framework Principal | Testing Framework | Librerías de Testing Adicionales | Build / Ejecución Tests |
|---|---|---|---|---|---|
| | | | | Memory | |
| **ms-user-management** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, AutoMapper | `dotnet test` via Docker Compose profile `tests` |
| **sg-ms-forgot-information** | C# / .NET 10 | ASP.NET Core Minimal APIs | **NUnit 4.x** | NUnit.Analyzers, Microsoft.NET.Test.Sdk, Microsoft.Extensions.Options, Microsoft.Extensions.Logging.Abstractions | `dotnet test` via Docker Compose profile `tests` |

**Nota:** Todos los servicios .NET usan **NUnit** como framework de testing principal (no xUnit ni MSTest). El servicio Java **ms-iam** usa **JUnit 5 (Jupiter)** con **Mockito** para mocking. Ningún servicio usa xUnit, MSTest, TestNG u otros frameworks.

---

## Detalle por Servicio

---

### 1. ms-fleet (Gestión de Flota)

**Ubicación:** `/backend/ms-fleet/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs
- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`)
- **Test Doubles:** In-Memory Repositories (Fakes), `FakeIamService`, `FakeGpsDeviceService`, `FakeTenantProvider`
- **Ejecución:** `dotnet test ./tests/ms-fleet.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-fleet.tests
```

**Resultado:** ✅ **39 tests pasados** en 313 ms

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `CreateBusServiceTests` | Crear bus (CU-FLEET-01) | 3 | Validación de datos válidos, placa duplicada, capacidad cero |
| `SearchBusesServiceTests` | Buscar buses (CU-FLEET-02) | 5 | Búsqueda por placa, marca, modelo, |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| | | | conductor; case-insensitive |
| `AssignDriverToBusServiceTests` | Asignar conductor a bus (CU-FLEET-03) | 4 | Asignación válida, bus inactivo, conductor ya asignado, bus ya tiene conductor |
| `GetBusAssignedToDriverServiceTests` | Obtener bus asignado a conductor (CU-FLEET-04) | 4 | Asignación activa, sin asignación, bus inactivo, asignación cerrada |
| `ListBusesBasicServiceTests` / `BusViewFieldsTests` | Listar buses con detalles (CU-FLEET-05) | 8 | Brand/Model/CampusId, estado inactivo, nombre conductor, detalle con brand/model |
| `ListBusesTenantFilterTests` | Filtrado multi-tenant (CU-FLEET-06) | 5 | Sin filtro, driver por campus, admin por escuela, superadmin sin filtro |
| `RouteGrpcServiceTests` | gRPC interno para ms-route (CU-FLEET-07) | 7 | GetBusAssignedToDriver (éxito, not found, invalid arg), GetBusDetails (con/sin conductor, not found, invalid arg) |
| `BusTests` | Dominio - Bus (CU-FLEET-08) | 2 | Estado activo por defecto, generación de ID |

***Código de Pruebas Representativas***

### CreateBusServiceTests.cs - Crear bus con validaciones:

```
[Test]
public async Task ExecuteAsync_WithValidData_ShouldReturnBusId()
{
    var repo = new InMemoryBusRepository();
    var gpsService = new FakeGpsDeviceService();
    var useCase = new CreateBusService(repo, gpsService);

    var busId = await useCase.ExecuteAsync(
        Guid.NewGuid(), DateTime.UtcNow.AddYears(1), Guid.NewGuid(),
        40, "ABC123", 1);

    Assert.That(busId, Is.Not.EqualTo(Guid.Empty));
}

[Test]
public async Task ExecuteAsync_WithDuplicatePlate_ShouldThrow()
{
    var repo = new InMemoryBusRepository();
```

```
    var gpsService = new FakeGpsDeviceService();
    var useCase = new CreateBusService(repo, gpsService);

    await useCase.ExecuteAsync(Guid.NewGuid(), DateTime.UtcNow.AddYears(1),
Guid.NewGuid(), 40, "ABC123", 1);

    Assert.ThrowsAsync<ArgumentException>(() =>
        useCase.ExecuteAsync(Guid.NewGuid(), DateTime.UtcNow.AddYears(1),
Guid.NewGuid(), 40, "ABC123", 1));
}
```

### SearchBusesServiceTests.cs - Búsqueda multi-criterio:

```
[Test]
public async Task Search_PlateShapedTerm_FiltersByPlate()
{
    var (service, mercedes, hino) = await CreateServiceAsync();
    var result = await service.SearchAsync("DEF456");
    Assert.That(result.Single().Id, Is.EqualTo(hino.Id));

    var lowerCase = await service.SearchAsync("abc123");
    Assert.That(lowerCase.Single().Id, Is.EqualTo(mercedes.Id));
}

[Test]
public async Task Search_GenericTerm_FiltersByBrandModelAndDriver()
{
    var (service, mercedes, hino) = await CreateServiceAsync();
    var byBrand = await service.SearchAsync("mercedes");
    Assert.That(byBrand.Single().Id, Is.EqualTo(mercedes.Id));
    var byModel = await service.SearchAsync("300");
    Assert.That(byModel.Single().Id, Is.EqualTo(hino.Id));
}
```

### AssignDriverToBusServiceTests.cs - Reglas de negocio:

```
[Test]
public async Task ExecuteAsync_WithDriverAlreadyAssigned_ShouldThrow()
{
    var busRepo = new InMemoryBusRepository();
    var assignmentRepo = new InMemoryDriverAssignmentRepository();
    var iamService = new FakeIamService();
    var useCase = new AssignDriverToBusService(busRepo, assignmentRepo, iamService);

    var bus1 = new Bus { Plate = "ABC123", Status = Status.Active };
    var bus2 = new Bus { Plate = "DEF456", Status = Status.Active };
    await busRepo.AddAsync(bus1);
    await busRepo.AddAsync(bus2);

    var profileId = Guid.NewGuid();
    await useCase.ExecuteAsync(bus1.Id, profileId);

    Assert.ThrowsAsync<InvalidOperationException>(() =>
        useCase.ExecuteAsync(bus2.Id, profileId));
}
```

***Evidencia de Ejecución (Docker Compose)***

```
Test run for /src/tests/ms-fleet.Tests/bin/Debug/net10.0/ms-fleet.Tests.dll
(.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    39, Skipped:     0, Total:    39, Duration: 313
ms
```

---

**2. ms-iam (Identidad y Acceso)**

**Ubicación:** `/backend/ms-iam/`

**Stack Tecnológico:**

- **Runtime:** Java 21 (OpenJDK / Eclipse Temurin)
- **Framework Web:** Spring Boot 3.2+ (Spring Web, Spring Security, Spring Data JPA)
- **Testing Framework: JUnit 5 (Jupiter)** - Anotaciones: `@Test`, `@ParameterizedTest`, `@ValueSource`, `@BeforeEach`, `@AfterEach`, `@DisplayName`
- **Mocking: Mockito 5.x** - `mock()`, `when()`, `verify()`, `ArgumentCaptor`, `verifyNoInteractions()`
- **Assertions:** JUnit 5 Assertions (`assertEquals`, `assertTrue`, `assertThrows`, `assertInstanceOf`) + AssertJ (fluido)
- **Spring Test:** `@SpringBootTest`, `@MockBean`, `@Autowired`, `MockMvc` (no usado en tests unitarios), `Testcontainers` (config)
- **Build Tool:** Maven 3.9+ (Surefire Plugin para tests unitarios)
- **Test Doubles:** Mockito mocks para repositorios, services, encoders, token providers

- **Ejecución:** `mvn test` dentro de contenedor `maven:3.9.6-eclipse-temurin-21`

**Resultado:** ✅ **57 tests pasados** en 1m 12s

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `LoginServiceTest` | Autenticación FR-2.1 | 6 | Credenciales válidas, expiraciones, email desconocido, contraseña incorrecta, cuenta inactiva, no emite tokens en fallo |
| `RefreshTokenServiceTest` | Renovación token FR-2.2 | 5 | Token válido, expiraciones, token expirado, perfil inexistente, cuenta inactiva |
| `ProfileDirectoryServiceTest` | Directorio de perfiles | 5 | Existencia activa/inactiva, nombre completo, solo nombre, perfil faltante (404) |
| `UpdateProfileStatusServiceTest` | Actualizar estado perfil | 4 | Aplica estado, idempotencia, estado desconocido, perfil faltante |
| `CreateProfileSchoolAssociationTest` | Asociación perfil-escuela | 4 | Usuarios regulares mantienen campus, admin crea asociación, mapeo repositorio, campus en columna |
| `InternalProfileServiceTest` | Perfil interno (gRPC) | 6 | Operaciones internas de perfil |
| `GlobalExceptionHandlerTest` | Manejo global de excepciones | 4 | Errores internos mantienen status, 401 credenciales, 401 refresh, 403 cuenta inactiva |
| `InternalApiKeyInterceptorTest` | Interceptor API Key gRPC | 4 | Key válida, key incorrecta, key faltante, key servidor vacía |
| `InternalProfileGrpcAdapterTest` | Adaptador gRPC interno | 9 | findProfileByEmail, phoneMatches, updatePassword, |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| | | | updateEmail, updatePhone |
| `ProfileGrpcAdapterTest` | Adaptador gRPC público | 4 | profileExists, getDriverName |
| `PersonStatusChangedConsumerTest` | Consumer Kafka estados | 2 | Swallows failures, mantiene consumidor corriendo |
| `KafkaConsumerConfigTest` | Config Kafka Consumer | 2 | Configuración de consumidor |
| `MsIamApplicationTests` | Test de contexto Spring | 1 | Carga de contexto |

***Código de Pruebas Representativas***

**LoginServiceTest.java - Autenticación (FR-2.1):**

```
@Test
void returnsBothTokensAndTheIdentityForValidCredentials() {
    expectValidCredentialsForActiveAccount();

    LoginResponseDto response = service.execute(new
LoginUseCase.Login("ana@school.edu", "secret"));

    assertEquals("access-token", response.accessToken());
    assertEquals("refresh-token", response.refreshToken());
    assertEquals(profileId, response.profileId());
    assertEquals(personId, response.personId());
    assertEquals("ana@school.edu", response.email());
    assertEquals((byte) 2, response.roleId());
    assertEquals(campusId, response.campusId());
    assertEquals(schoolId, response.schoolId());
}

@Test
void rejectsAnInactiveAccount() {
    when(authenticationRepository.findByEmail("ana@school.edu"))
        .thenReturn(Optional.of(accountWithStatus("Inactive")));
    when(passwordEncoder.matches("secret", "hashed")).thenReturn(true);

    assertThrows(LoginService.AccountInactiveException.class,
        () -> service.execute(new LoginUseCase.Login("ana@school.edu", "secret")));
}
```

**RefreshTokenServiceTest.java - Renovación token (FR-2.2):**

```
@Test
void issuesANewTokenPairForAValidRefreshToken() {
    expectAValidRefreshToken();
    when(authenticationRepository.findByProfileId(profileId))
        .thenReturn(Optional.of(accountWithStatus("Active")));
```

```
    when(tokenProvider.generatedAccessToken(any(), any(), any(), any(), any(), any(),
any(), any(), any(), any()))
        .thenReturn("new-access-token");
    when(tokenProvider.generatedRefreshToken(any(), any(), any(), any(), any(), any(),
any(), any()))
        .thenReturn("new-refresh-token");

    RefreshTokenUseCase.RefreshResponse response =
        service.execute(new RefreshTokenUseCase.Refresh("valid-refresh"));

    assertEquals("new-access-token", response.accessToken());
    assertEquals("new-refresh-token", response.refreshToken());
}

@Test
void rejectsAnExpiredRefreshToken() {
    when(tokenProvider.parseRefreshToken("stale-refresh"))
        .thenReturn(claimsExpiringAt(Instant.now().minusSeconds(1)));

    assertThrows(RefreshTokenService.InvalidRefreshTokenException.class,
        () -> service.execute(new RefreshTokenUseCase.Refresh("stale-refresh")));

    verifyNoInteractions(authenticationRepository);
}
```

### InternalApiKeyInterceptorTest.java - Seguridad gRPC:

```
@Test
void validKeyReachesTheService() {
    var interceptor = new InternalApiKeyInterceptor("test-key");
    var headers = headersWith("test-key");

    interceptor.interceptCall(call, headers, next);

    verify(next).startCall(call, headers);
    verify(call, never()).close(any(), any());
}

@Test
void wrongKeyIsRejectedUnauthenticated() {
    var interceptor = new InternalApiKeyInterceptor("test-key");
    interceptor.interceptCall(call, headersWith("other-key"), next);

    ArgumentCaptor<Status> status = ArgumentCaptor.forClass(Status.class);
    verify(call).close(status.capture(), any());
    assertEquals(Status.UNAUTHENTICATED.getCode(), status.getValue().getCode());
    verifyNoInteractions(next);
}
```

### *Evidencia de Ejecución (Maven)*

```
[INFO] Tests run: 57, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:12 min
[INFO] Finished at: 2026-10-25T03:07:18Z
```

---

### 3. ms-notification (Notificaciones e Incidentes)

**Ubicación:** `/backend/ms-notification/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs
- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`, `Is.EquivalentTo()`)
- **ORM/DB:** Entity Framework Core 9 con **In-Memory Database** para tests de integración (`UseInMemoryDatabase()`)
- **Test Doubles:** Fakes (`RecordingEventPublisher`, `FakeRouteDirectory`, `FakeFamilyDirectory`, `FakeSchoolDirectory`, `FakeStudentDirectory`, `FixedTimeProvider`, `FakeConfiguration`), `NullLogger<T>`
- **Mapeo:** AutoMapper con `MapperConfiguration` + `IncidentProfile`
- **Ejecución:** `dotnet test ./tests/ms-notification.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-notification.tests
```

**Resultado:** ✅ **56 tests pasados** en 3 s

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `ReportIncidentServiceTests` | Reportar incidente FR-8.1, FR-8.2 | 21 | Registra alerta, notifica escuela y acudientes, omite estudiantes bajados, deduplica padres, publica evento, rechaza sin viaje activo, tipo desconocido, descripción larga, resilencia directorios caídos |
| `ProcessScanServiceTests` | Procesar escaneo QR FR-4.2, FR-4.3, FR-5.1 | 39 | Subida/bajada persiste y publica, idempotencia, validación ruta/horario/día, token QR inválido, conductor sin viaje, estudiante sin ruta, lookup guardian falla |
| `GetIncidentByIdServiceTests` | Obtener incidente por ID | 7 | Incidente desconocido, tipo fuera de catálogo, incidente existente, resuelto (status Open), mapeo DTO |
| `GetAlertByIdServiceTests` | Obtener alerta por ID | 4 | Alerta existente, inexistente, mapeo DTO |
| `NotifyIncidentReportedServiceTests` | Push notificación incidente FR-8.2 | 10 | Push a dispositivos destinatarios, deduplica tokens/perfiles, sin dispositivos no push, fallback descripción vacía |
| `AlertDescriptionBui` | Builder descripción | 5 | Al bajar (indica |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| lderTests | alertas | | bajada+ubicación), al subir, sin dirección, sin nombre estudiante, texto largo con límite |

***Código de Pruebas Representativas***

**ReportIncidentServiceTests.cs - Reporte incidente (FR-8.1, FR-8.2):**

```
[Test]
public async Task Report_NotifiesTheSchoolAndTheGuardiansOfTheStudentsOnBoard()
{
    Board(_studentA, BoardingType.ON_BOARD, DateTime.UtcNow.AddMinutes(-10));
    _family.ByStudent[_studentA] = new[] { _parentA };

    var response = await _service.ExecuteAsync(_driverId, Request());

    var recipients = await _context.AlertRecipients.Select(r =>
r.ProfileId).ToListAsync();
    Assert.That(recipients, Is.EquivalentTo(new[] { _schoolAdmin, _parentA }));
    Assert.That(response.RecipientsNotified, Is.EqualTo(2));
}

[Test]
public async Task Report_SkipsStudentsWhoAlreadyGotOffTheBus()
{
    Board(_studentA, BoardingType.ON_BOARD, DateTime.UtcNow.AddMinutes(-10));
    Board(_studentB, BoardingType.ON_BOARD, DateTime.UtcNow.AddMinutes(-9));
    Board(_studentB, BoardingType.OFF_BOARD, DateTime.UtcNow.AddMinutes(-2));
    _family.ParentProfileIds = new[] { _parentA };

    var response = await _service.ExecuteAsync(_driverId, Request());

    Assert.That(response.StudentsOnBoard, Is.EqualTo(1));
    Assert.That(_family.Queried, Is.EquivalentTo(new[] { _studentA }));
}

[Test]
public async Task Report_StillNotifiesTheGuardiansWhenTheSchoolDirectoryIsDown()
{
    Board(_studentA, BoardingType.ON_BOARD, DateTime.UtcNow.AddMinutes(-10));
    _family.ParentProfileIds = new[] { _parentA };
    _school.ThrowOnRead = new InvalidOperationException("ms-school-management down");

    var response = await _service.ExecuteAsync(_driverId, Request());

    var recipients = await _context.AlertRecipients.Select(r =>
r.ProfileId).ToListAsync();
    Assert.That(recipients, Is.EquivalentTo(new[] { _parentA }));
    Assert.That(response.RecipientsNotified, Is.EqualTo(1));
}
```

**ProcessScanServiceTests.cs - Escaneo QR (FR-4.2, FR-4.3, FR-5.1):**

```
[Test]
public async Task BoardingPersistsTheBoardingAndTheAlertAndPublishesTheEvent()
{
    var response = await Scan(BoardingType.ON_BOARD);

    Assert.That(response.BoardingId, Is.Not.EqualTo(Guid.Empty));
    Assert.That(response.AlertId, Is.Not.EqualTo(Guid.Empty));

    Assert.That(_context.Boardings.Count(), Is.EqualTo(1));
    var boarding = _context.Boardings.Single();
    Assert.That(boarding.ProfileId, Is.EqualTo(_studentId));
    Assert.That(boarding.RouteExecutionId, Is.EqualTo(_tripId));
    Assert.That(boarding.RouteStopId, Is.EqualTo(_routeStopId));
    Assert.That(boarding.BoardingType, Is.EqualTo(BoardingType.ON_BOARD));

    Assert.That(_context.Alerts.Count(), Is.EqualTo(1));
    var alert = _context.Alerts.Single();
    Assert.That(alert.AlertTypeId, Is.EqualTo((byte)1));
    Assert.That(alert.Description, Does.Contain("Libertad").And.Contain("Calle 12 #34-
56"));

    Assert.That(_publisher.Published, Has.Count.EqualTo(1));
    Assert.That(_publisher.Published[0].Topic, Is.EqualTo("student.scanned"));
}

[Test]
public async Task ARepeatedOnBoardScanReusesTheBoardingAndDoesNotNotifyAgain()
{
    var first = await Scan(BoardingType.ON_BOARD);
    var second = await Scan(BoardingType.ON_BOARD);

    Assert.That(second.BoardingId, Is.EqualTo(first.BoardingId));
    Assert.That(second.AlertId, Is.EqualTo(first.AlertId));
    Assert.That(second.AlreadyProcessed, Is.True);
    Assert.That(first.AlreadyProcessed, Is.False);
    Assert.That(_context.Boardings.Count(), Is.EqualTo(1));
    Assert.That(_context.Alerts.Count(), Is.EqualTo(1));
    Assert.That(_publisher.Published, Has.Count.EqualTo(1));
}
```

### *Evidencia de Ejecución (Docker Compose)*

```
Test run for /src/tests/ms-notification.Tests/bin/Debug/net10.0/ms-
notification.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    56, Skipped:     0, Total:    56, Duration: 3 s
```

---

**4. ms-route (Rutas y Viajes)**

**Ubicación:** `/backend/ms-route/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs
- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`, `Is.EquivalentTo()`, `Is.Null`)
- **Test Doubles:** In-Memory Repositories (`InMemoryRouteRepository`, `InMemoryRouteBusAssignmentRepository`, `InMemoryRouteStopRepository`, `InMemoryStopRepository`, `InMemoryRouteScheduleRepository`, `InMemorySchoolCampusRepository`, `InMemoryRouteStudentAssignmentRepository`), Fakes (`FakeFleetService`, `FakeTenantProvider`, `FixedTimeProvider`, `TestMapper`)
- **Mapeo:** `TestMapper.Create()` (AutoMapper simplificado para tests)
- **Lógica Temporal:** `FixedTimeProvider` para control determinista de tiempo (TimeOnly, DateTimeOffset, zonas horarias Colombia UTC-5)
- **Ejecución:** `dotnet test ./tests/ms-route.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-route.tests
```

**Resultado:** ✅ **74 tests pasados** en 1 s

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `CreateRouteServiceTests` | Crear ruta | 3 | Ruta válida, nombre duplicado, nombre vacío |
| `GetRouteServiceTests` | Obtener ruta con bus y paradas | 4 | Ruta existente, inexistente, con bus/paradas, sin bus |
| `ListRouteServiceTests` | Listar rutas | 3 | Con rutas, sin rutas, incluye destino y conteo paradas |
| `AssignBusToRouteServiceTests` | Asignar bus a ruta | 4 | Ruta y bus libres, ruta inexistente, ruta ya tiene bus, bus en otra ruta |
| `AssignStudentToRouteServiceTests` | Asignar estudiante a ruta | 5 | Asignación válida, estudiante ya asignado, ruta inexistente, parada inexistente, estudiante inactivo |
| `GetStudentRouteServiceTests` | Obtener ruta del estudiante | 8 | Ruta activa con paradas, estudiante sin ruta, ruta inactiva, etc. |
| `GetDriverRouteTodayServiceTests` | Ruta del conductor hoy | 25 | Ventana activa (15min antes), programada (antes), día anterior, después ventana, fin de semana, sin bus, sin asignación, ruta inactiva, sin horarios, ruta sin hora fin, respaldo 1hr, sin horario configurado |
| `GetCurrentRouteServiceTests` | Ruta actual en viaje | 5 | Viaje activo, sin viaje, viaje finalizado |
| `GetCurrentTripServiceTests` | Viaje actual | 4 | Viaje en progreso, sin viaje, viaje finalizado |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| GetRouteSuggestionServiceTests | Sugerencia de ruta | 4 | Con sugerencia, sin sugerencia |
| UpdateRouteServiceTests | Actualizar ruta | 4 | Actualización válida, ruta inexistente, nombre duplicado |
| UnassignBusFromRouteServiceTests | Desasignar bus | 3 | Desasignación válida, sin asignación, asignación inactiva |
| DeleteRouteServiceTests | Eliminar ruta | 3 | Eliminación válida, ruta inexistente, ruta con asignaciones |
| CreateRouteScheduleServiceTests | Crear horario ruta | 4 | Horario válido, ruta inexistente, asignación inexistente, duplicado |
| ListRouteSchedulesServiceTests | Listar horarios | 3 | Con horarios, sin horarios, filtra por asignación |
| SearchRouteServiceTests | Buscar rutas | 4 | Término vacío, por nombre, sin coincidencias |
| SearchStopServiceTests | Buscar paradas | 3 | Término vacío, por nombre, sin coincidencias |
| GetStudentStopOnRouteServiceTests | Parada del estudiante | 4 | Parada existente, estudiante sin ruta, ruta sin parada |

### *Código de Pruebas Representativas*

#### CreateRouteServiceTests.cs - Crear ruta:

```
[Test]
public async Task ExecuteAsync_ConRutaValida_CreaRuta()
{
    var repo = new InMemoryRouteRepository();
    var service = new CreateRouteService(repo, TestMapper.Create());

    var result = await service.ExecuteAsync(new RouteRequestDto
    {
        CampuseId = Guid.NewGuid(),
        Name = "R-01",
        TargetSector = "Norte"
    });

    Assert.That(repo.Routes, Has.Exactly(1).Items);
    Assert.That(result.Name, Is.EqualTo("R-01"));
}
```

```
[Test]
public async Task ExecuteAsync_ConNombreDuplicado_Rechaza()
{
    var repo = new InMemoryRouteRepository();
    repo.Routes.Add(new RouteModel { Id = Guid.NewGuid(), Name = "R-01", TargetSector
= "Norte", Status = Status.Active });
    var service = new CreateRouteService(repo, TestMapper.Create());

    Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.ExecuteAsync(new RouteRequestDto
        {
            CampuseId = Guid.NewGuid(),
            Name = "R-01",
            TargetSector = "Sur"
        }));
}
```

**GetDriverRouteTodayServiceTests.cs - Ruta del conductor hoy (lógica temporal compleja):**

```
[Test]
public async Task ExecuteAsync_DentroDeLaVentana_RetornaActiveConParadas()
{
    AddStopToRoute();
    var service = CreateService(Local(Monday, new TimeOnly(7, 0)));

    var result = await service.ExecuteAsync(Guid.NewGuid());

    Assert.That(result.Status, Is.EqualTo(DriverRouteStatus.Active));
    Assert.That(result.Route, Is.Not.Null);
    Assert.That(result.Route.Stops, Has.Exactly(1).Items);
    Assert.That(result.NextOccurrence, Is.Null);
}

[Test]
public async Task ExecuteAsync_QuinceMinutosAntesDeLaSalida_RetornaActive()
{
    var service = CreateService(Local(Monday, new TimeOnly(5, 45)));
    var result = await service.ExecuteAsync(Guid.NewGuid());
    Assert.That(result.Status, Is.EqualTo(DriverRouteStatus.Active));
}

[Test]
public async Task
ExecuteAsync_DespuesDeLaVentana_RetornaScheduledParaLaProximaSalida()
{
    var service = CreateService(Local(Monday, new TimeOnly(11, 0)));
    var result = await service.ExecuteAsync(Guid.NewGuid());

    Assert.That(result.Status, Is.EqualTo(DriverRouteStatus.Scheduled));
    Assert.That(result.NextOccurrence, Is.EqualTo(Local(Monday.AddDays(7),
RouteStart)));
}
```

**AssignBusToRouteServiceTests.cs - Reglas de asignación:**

```
[Test]
public async Task ExecuteAsync_ConBusAsignadoAOtraRuta_Rechaza()
{
    var routeId = Guid.NewGuid();
    var otherRouteId = Guid.NewGuid();
    var busId = Guid.NewGuid();

    var routes = new InMemoryRouteRepository();
    routes.Routes.Add(new RouteModel { Id = routeId, Name = "R-01", Status =
Status.Active });

    var assignments = new InMemoryRouteBusAssignmentRepository();
    assignments.Assignments.Add(new RouteBusAssignment
    {
        Id = Guid.NewGuid(),
        RouteId = otherRouteId,
        BusId = busId,
        Status = Status.Active
    });

    var service = new AssignBusToRouteService(assignments, routes,
TestMapper.Create());

    Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.ExecuteAsync(routeId, busId));
}
```

***Evidencia de Ejecución (Docker Compose)***

```
Test run for /src/tests/ms-route.Tests/bin/Debug/net10.0/ms-route.Tests.dll
(.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    74, Skipped:     0, Total:    74, Duration: 1 s
```

---

## 5. ms-school-management (Gestión de Escuelas y Sedes)

**Ubicación:** `/backend/ms-school-management/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs

- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`, `[TestCase]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`, `Is.Empty`, `Is.InstanceOf<T>()`)
- **Mapeo:** AutoMapper 13+ con `MapperConfiguration` + Profiles (`SchoolProfile`, `CampusMapper`, `AdminSchoolMapper`) + `IMapper` inyectado via `ServiceCollection`
- **Caching:** `Microsoft.Extensions.Caching.Memory.MemoryCache` para `TenantSchoolResolver`
- **Test Doubles:** Fakes (`FakeSchoolRepository`, `FakeSchoolAdminRepository`, `FakeCampusRepository`, `FakeTenantProvider`, `NoFilterTenantSchoolResolver`, `Repository` inline para `SchoolCentralCampusTests`)
- **Controller Tests:** `Microsoft.AspNetCore.Mvc` (`OkObjectResult`, `NotFoundResult`) para tests de `CampusController`
- **Ejecución:** `dotnet test ./tests/ms-school-management.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-school-management.tests
```

**Resultado:** ✅ **22 tests pasados** en 817 ms

### *Casos de Uso / Historias de Usuario Cubiertas*

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `GetSchoolServiceTests` | Obtener escuela con ciudad | 3 | Con ciudad (rellena CityName), ciudad sin registro (CityName vacío), escuela inexistente (null) |
| `SearchSchoolsServiceTests` | Buscar escuelas | 4 | Término vacío, por nombre, por dirección, sin coincidencias |
| `ListSchoolsServiceTests` | Listar escuelas con filtrado tenant | 5 | Sin token (todos), admin resuelve por SchoolAdmin (prioridad), admin sin registro usa claim, student con campus retorna escuela dueña, student sin campus no filtra |
| `SchoolCentralCampusTests` | Sede central en creación/actualización | 8 | Update desactiva solo removidas explícitas, update sin removidas no desactiva omitidas, rechaza IDs vacíos/conflictivos, creación con sedes persiste central, creación simple persiste central, cliente existente provee central sin duplicar |

### *Código de Pruebas Representativas*

#### GetSchoolServiceTests.cs - Obtener escuela:

```
[Test]
public async Task ExecuteAsync_ConCiudad_RellenaCityName()
{
    var schoolId = Guid.NewGuid();
    var cityId = Guid.NewGuid();
    var repository = new FakeSchoolRepository(NewSchool(schoolId, cityId))
    {
        CityNames = { [cityId] = "Bogota" }
    };
    var service = new GetSchoolService(repository, CreateMapper());
```

```
    var result = await service.ExecuteAsync(schoolId);

    Assert.That(result, Is.Not.Null);
    Assert.That(result.CityName, Is.EqualTo("Bogota"));
    Assert.That(result.CityId, Is.EqualTo(cityId));
}
```

### ListSchoolsServiceTests.cs - Filtrado multi-tenant:

```
[Test]
public async Task ExecuteAsync_Admin_ResuelvePorSchoolAdminConPrioridad()
{
    // El registro SchoolAdmin (Sabana) debe ganar sobre el claim (Oriente).
    var service = CreateService(new FakeTenantProvider(
        shouldFilter: true, roleId: 1, profileId: ProfileId, schoolId: OrienteId));

    var result = await service.ExecuteAsync();

    var schoolItems = result;
    Assert.That(schoolItems, Has.Exactly(1).Items);
    var school = schoolItems.Single();
    Assert.That(school.Name, Is.EqualTo("Institucion Educativa Sabana"));
}

[Test]
public async Task ExecuteAsync_StudentConCampus_RetornaColegioDuenoDeSuSede()
{
    var service = CreateService(new FakeTenantProvider(
        shouldFilter: true, roleId: 2, campusId: CampusSabanaId));

    var result = await service.ExecuteAsync();

    var schoolItems = result;
    Assert.That(schoolItems, Has.Exactly(1).Items);
    var school = schoolItems.Single();
    Assert.That(school.Id, Is.EqualTo(SabanaId));
}
```

### SchoolCentralCampusTests.cs - Sede central:

```
[Test]
public async Task UpdateDeactivatesOnlyExplicitlyRemovedCampuses()
{
    var repository = new Repository();
    var schoolId = Guid.NewGuid();
    var retainedId = Guid.NewGuid();
    var removedId = Guid.NewGuid();
    await repository.SaveWithCampusesAsync(new SchoolModel { Id = schoolId }, []);
    var request = new SchoolWithCampusesRequestDto(Guid.NewGuid(), [], "Test school",
        "Central street 123", null, null, 3001234567, "school@example.invalid", null,
"Primaria",
        [new SchoolCampusRequestDto(retainedId, "North", "North street 456", null,
null)],
        RemovedCampusIds: [removedId, removedId]);

    await new UpdateSchoolWithCampusesService(repository,
Mapper()).ExecuteAsync(schoolId, request);
```

```
    Assert.That(repository.Campuses.Count, Is.EqualTo(2));
    Assert.That(repository.Campuses.Single(c => c.Id == retainedId).Status,
Is.EqualTo(Status.Active));
    Assert.That(repository.Campuses.Single(c => c.Id == removedId).Status,
Is.EqualTo(Status.Inactive));
}

[TestCase(false)]
[TestCase(true)]
public async Task WithCampusesPersistsCentralAndOptionalAdditionalCampus(bool
additional)
{
    var repository = new Repository();
    var request = new SchoolWithCampusesRequestDto(Guid.NewGuid(), [], "Test school",
        "Central street 123", 2.9m, -75.2m, 3001234567, "school@example.invalid",
null, "Primaria",
        additional ? [new SchoolCampusRequestDto(null, "North", "North street 456",
null, null)] : []);
    var result = await new CreateSchoolWithCampusDetailsService(repository,
Mapper()).ExecuteAsync(request);
    Assert.That(result.Campuses.Count, Is.EqualTo(additional ? 2 : 1));
    var central = repository.Campuses[0];
    Assert.That(central.Name, Is.EqualTo("Sede central"));
    Assert.That(central.SchoolId, Is.EqualTo(result.Id));
}
```

### CampusControllerTests.cs - Controller REST:

```
[Test]
public async Task Get_CampusExistente_RetornaOkConElDetalle()
{
    var id = Guid.NewGuid();
    var schoolId = Guid.NewGuid();
    var campus = new SchoolCampus
    {
        Id = id,
        SchoolId = schoolId,
        Name = "Sede norte",
        Address = "Calle 10 # 20-30",
        Status = Status.Active
    };

    var result = await Controller(campus).Get(id, CancellationToken.None);

    Assert.That(result, Is.InstanceOf<OkObjectResult>());
    var dto = (CampusDetailDto)((OkObjectResult)result).Value!;
    Assert.That(dto.Id, Is.EqualTo(id));
    Assert.That(dto.SchoolId, Is.EqualTo(schoolId));
    Assert.That(dto.Name, Is.EqualTo("Sede norte"));
}
```

### *Evidencia de Ejecución (Docker Compose)*

```
Test run for /src/tests/ms-school-management.Tests/bin/Debug/net10.0/ms-school-
management.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.
```

```
Passed!  - Failed:     0, Passed:    22, Skipped:     0, Total:    22, Duration: 817
ms
```

---

### 6. ms-user-management (Gestión de Usuarios y Familias)

**Ubicación:** `/backend/ms-user-management/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs
- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`, `Is.Empty`, `Is.True`, `Is.False`, `Is.Null`, `ThrowsAsync<T>()`)
- **Mapeo:** AutoMapper 13+ con `MapperConfiguration` + `PersonProfile` + `IMapper` via `ServiceCollection`
- **Test Doubles:** In-Memory Repositories (`InMemoryFamilyRepository`, `InMemoryDriverLicenseRepository`, `InMemoryPersonRepository`, `InMemoryPersonProfileReader`, `InMemoryPersonLicenseReader`), Fakes (`TenantFakes.NoFilter()`, `FakePersonSearchRepository`, `RecordingPersonRepository`, `InMemoryDriverLicenseRepository`)

- **Patrones de Búsqueda:** Strategy Pattern para búsquedas (`IPersonSearchStrategy` → `NameSearchStrategy`, `PhoneSearchStrategy`)
- **Ejecución:** `dotnet test ./tests/ms-user-management.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-user-management.tests
```

**Resultado:** ✅ **66 tests pasados** en 1 s

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `SearchPersonServiceTests` | Buscar personas | 5 | Término vacío, con licencia (rellena licencia/vencimiento), rellena ProfileId, filtra por rol |
| `RegisterFamilyServiceTests` | Registrar familia | 5 | Familia válida guarda familia y vínculos, miembro ya asociado rechaza, perfiles duplicados en lote rechaza, lote vacío rechaza, sin nombre rechaza |
| `DeleteFamilyServiceTests` | Eliminar familia | 2 | Elimina familia y miembros, familia inexistente lanza error |
| `UpdateFamilyServiceTests` | Actualizar familia | 6 | Actualización válida, familia inexistente, nombre vacío, miembro nuevo, miembro existente, remover |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| | | | miembro |
| `GetFamilyServiceTests` | Obtener familia | 4 | Familia existente, inexistente, con miembros, sin miembros |
| `ListFamilyServiceTests` | Listar familias | 3 | Con familias, sin familias, paginación |
| `SearchFamilyServiceTests` | Buscar familias | 3 | Término vacío, por nombre, sin coincidencias |
| `DriverLicenseServiceTests` | Licencia de conducir | 8 | Crear licencia válida, sin ProfileId rechaza, número duplicado rechaza, sin fecha expiración rechaza, listar por ProfileId, actualizar y eliminar, actualizar/get inexistente |
| `DriverLicenseViewTests` | Vista licencia conductor | 3 | Vista con licencia, sin licencia, múltiples licencias |
| `ChangePersonStatusServiceTests` | Cambiar estado persona | 4 | Cambio válido, persona inexistente, mismo estado (idempotente), estado inválido |
| `PersonPhoneTests` / `PersonPhoneContractTests` | Teléfono persona | 6 | Actualización teléfono, validación formato, contrato |
| `PersonUpdateTests` | Actualizar persona | 5 | Actualización válida, persona inexistente, email duplicado, validaciones |
| `PersonDeleteTests` | Eliminar persona | 3 | Eliminación válida, persona inexistente, cascada familia |
| `UserCampusCreationTests` | Creación usuario con campus | 4 | Estudiante con campus, conductor con campus, validaciones |
| `RoleFilteredListTests` | Listas filtradas por rol | 4 | Filtro por rol estudiante, conductor, padre, admin |
| `ApiErrorsTests` | Manejo errores API | 4 | Mapeo excepciones a HTTP status codes |

***Código de Pruebas Representativas***

**RegisterFamilyServiceTests.cs - Registrar familia:**

```
[Test]
public async Task ExecuteAsync_ConFamiliaValida_GuardaFamiliaYVinculos()
{
    var repo = new InMemoryFamilyRepository();
    var service = new RegisterFamilyService(repo);
    var parent = Guid.NewGuid();
    var child = Guid.NewGuid();

    var (familyId, members) = await service.ExecuteAsync(
        "Perez", "Ruta 5",
        new List<(Guid, RelationType)>
        {
            (parent, RelationType.Parent),
            (child, RelationType.Student)
        });

    Assert.That(repo.Families.Where(f => f.Id == familyId && f.Name == "Perez"),
Has.Exactly(1).Items);
    Assert.That(members.Count, Is.EqualTo(2));
    Assert.That(repo.Members.Count(m => m.FamilyId == familyId), Is.EqualTo(2));
    Assert.That(repo.Members.Any(m => m.ProfileId == parent && m.RelationType ==
RelationType.Parent), Is.True);
    Assert.That(repo.Members.Any(m => m.ProfileId == child && m.RelationType ==
RelationType.Student), Is.True);
}

[Test]
public async Task ExecuteAsync_ConMiembroYaAsociado_Rechaza()
{
    var repo = new InMemoryFamilyRepository();
    var taken = Guid.NewGuid();
    repo.Members.Add((Guid.NewGuid(), Guid.NewGuid(), taken, RelationType.Student));
    var service = new RegisterFamilyService(repo);

    var ex = Assert.ThrowsAsync<ProfileAlreadyInFamilyException>(() =>
        service.ExecuteAsync("Perez", null,
            new List<(Guid, RelationType)> { (taken, RelationType.Student) }));

    Assert.That(ex.Message, Does.Contain("pertenece a otra familia"));
    Assert.That(repo.Families, Is.Empty);
}
```

### DriverLicenseServiceTests.cs - Licencia de conducir:

```
[Test]
public async Task Create_ConDatosValidos_GuardaLicenciaActiva()
{
    var repo = new InMemoryDriverLicenseRepository();
    var service = new CreateDriverLicenseService(repo);
    var profileId = Guid.NewGuid();

    var id = await service.ExecuteAsync(ValidRequest(profileId));

    var storedItems = repo.Licenses;
    Assert.That(storedItems, Has.Exactly(1).Items);
    var stored = storedItems.Single();
    Assert.That(stored.Id, Is.EqualTo(id));
    Assert.That(stored.ProfileId, Is.EqualTo(profileId));
    Assert.That(stored.LicenseNumber, Is.EqualTo("ABC12345"));
    Assert.That(stored.Status, Is.EqualTo(Status.Active));
}
```

```
[Test]
public async Task Create_ConNumeroDeLicenciaRepetido_Rechaza()
{
    var repo = new InMemoryDriverLicenseRepository();
    var service = new CreateDriverLicenseService(repo);
    await service.ExecuteAsync(ValidRequest());

    var ex = Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.ExecuteAsync(ValidRequest()));

    Assert.That(ex.Message, Does.Contain("already exists"));
    Assert.That(repo.Licenses, Has.Exactly(1).Items);
}
```

### SearchPersonServiceTests.cs - Búsqueda con licencia y rol:

```
[Test]
public async Task Search_ConLicencia_RellenaLicenciaYVencimiento()
{
    var personId = Guid.NewGuid();
    var repo = new FakePersonSearchRepository();
    repo.People.Add(Driver(personId, "Ana", "ana@mail.com"));

    var reader = new InMemoryPersonLicenseReader();
    reader.LicensesByPerson[personId] = new DriverLicense
    {
        ProfileId = Guid.NewGuid(),
        LicenseNumber = "ABC12345",
        LicenseExpirationDate = new DateOnly(2030, 12, 31),
        Status = default
    };

    var service = new SearchPersonService(
        new IPersonSearchStrategy[] { new NameSearchStrategy(repo) },
        reader,
        new InMemoryPersonProfileReader(),
        TenantFakes.NoFilter(new InMemoryPersonProfileReader()),
        CreateMapper());

    var result = (await service.SearchAsync("ana")).ToList();

    var singleItems = result;
    Assert.That(singleItems, Has.Exactly(1).Items);
    var single = singleItems.Single();
    Assert.That(single.LicenseNumber, Is.EqualTo("ABC12345"));
    Assert.That(single.LicenseExpirationDate, Is.EqualTo(new DateOnly(2030, 12, 31)));
}
```

### *Evidencia de Ejecución (Docker Compose)*

```
Test run for /src/tests/ms-user-management.Tests/bin/Debug/net10.0/ms-user-
management.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    66, Skipped:     0, Total:    66, Duration: 1 s
```

---

**7. sg-ms-forgot-information (Recuperación de Información)**

**Ubicación:** `/backend/sg-ms-forgot-information/`

**Stack Tecnológico:**

- **Runtime:** .NET 10 (C# 13)
- **Framework Web:** ASP.NET Core Minimal APIs
- **Testing Framework: NUnit 4.x** (attributes: `[Test]`, `[TestFixture]`, `[SetUp]`, `[TearDown]`, `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`, `[TestCase]`)
- **Assertions:** NUnit Constraints Model (`Assert.That()`, `Is.EqualTo()`, `Is.Not.Null`, `Has.Count.EqualTo()`, `Does.Contain()`, `Is.Empty`, `Is.True`, `Is.False`, `Is.Null`, `ThrowsAsync<T>()`, `Is.TypeOf<T>()`)
- **Test Doubles:** Fakes especializados:
  - `FakeVerificationRequestRepository` - repositorio en memoria para códigos OTP
  - `FakeIdentityDirectoryClient` - cliente gRPC fake para ms-iam
  - `FakeEmailSender` - capturador de emails enviados
  - `FakeSmsVerificationService` - simulador de Twilio Verify con `SmsCheckResult` (Approved/Rejected/Expired)
  - `ManualTimeProvider` - control manual de tiempo para tests de expiración

- `FakeSmtpServer` - servidor SMTP in-process real (TcpListener) para tests de integración de email

- **OTP/Hashing:** `SecretHasher` (PBKDF2/HMAC-SHA256 con pepper), `OtpCodeGenerator` (6 dígitos), `OtpOptions` (configuración: MaxAttempts, ExpirationMinutes, MaxRequestsPerWindow, RequestWindowMinutes, ResetTokenExpirationMinutes)
- **Email:** `TransactionalEmails` (HTML + Text), `EmailBrandingOptions`, `GuardianEmailTemplate`, `EmailLogMask` (masking PII en logs)
- **Mapeo de Errores Twilio:** `TwilioErrorMapper` (códigos 60200, 60203, 60202, 20404, 20003 → Exceptions controladas)
- **Ejecución:** `dotnet test ./tests/ms-forgot-information.Tests/ --nologo` dentro de contenedor `mcr.microsoft.com/dotnet/sdk:10.0`

**Archivo Docker Compose de Tests:** `docker-compose.yml` (perfil `tests`)

**Comando de ejecución:**

```
docker compose --profile tests run --rm ms-forgot-information.tests
```

**Resultado:** ✅ **62 tests pasados** en 1 s

***Casos de Uso / Historias de Usuario Cubiertas***

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| `ForgotPasswordServiceTests` | Olvidé contraseña | 2 | Cuenta no encontrada (sin código), email existe |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| | | | (emite código) |
| `PhoneChangeTests` | Cambio de teléfono | 31 | Flujo completo identidad+cambio, SMS a teléfono exacto, teléfono inválido no llega a Twilio, normalización teléfono, requiere token identidad, código aprobado actualiza teléfono, código rechazado no actualiza, reintentos bloquean challenge, teléfono diferente al verificado rechaza, check sin verificación rechaza, fallo Twilio error controlado, teléfono no marcado si IAM falla, rate limit SMS identidad, SMS identidad solo al teléfono actual, no envía si teléfono no coincide, no envía para email desconocido, teléfono inválido nunca a Twilio, código rechazado sin token, código verificado contra teléfono del SMS, mapeo errores Twilio |
| `VerificationCodeServiceTests` | Servicio códigos verificación | 10 | Emite código por email, verifica código correcto, código incorrecto lanza excepción, bloquea tras MaxAttempts, expiración lanza CodeExpiredException, rate limit IssueAsync, código consumido no reutilizable |
| `SmtpEmailSenderTests` | Envío email SMTP | 3 | Entrega a servidor SMTP sin credenciales, envuelve fallos conexión en NotificationDeliveryException, falla si SMTP no configurado |
| `TransactionalEmailsTests` | Emails transaccionales | 6 | Todo propósito renderiza código en HTML/texto, declara light color |

| Test Class | Funcionalidad | Tests | Descripción |
|---|---|---|---|
| | | | scheme, valores dinámicos HTML encoded, links/imágenes opcionales solo si configurados, texto plano contiene solo un número de 6 dígitos |
| `EmailDeliveryAndResetTokenTests` | Entrega email y token reset | 5 | Flujo entrega email + token reset |
| `IdentityDirectoryGrpcClientTests` | Cliente gRPC directorio identidad | 5 | Operaciones gRPC contra ms-iam |

***Código de Pruebas Representativas***

**PhoneChangeTests.cs - Cambio teléfono (flujo completo):**

```
[Test]
public async Task Valid_request_asks_twilio_to_send_the_sms_to_exactly_the_new_phone()
{
    var token = await ProveIdentityAsync();
    var emailsBefore = _email.Sent.Count;

    await RequestStep().ExecuteAsync(new RequestPhoneVerificationDto(Email, token,
NewPhone), "127.0.0.1", CancellationToken.None);

    Assert.That(_sms.StartedFor, Is.EqualTo(new[] { CurrentPhone, NewPhone }));
    Assert.That(_email.Sent, Is.Empty); // el flujo es solo SMS: sin SMTP
    Assert.That(_identity.LastPhoneUpdate, Is.Null); // solicitar código nunca cambia
el teléfono
}

[Test]
public async Task Approved_code_updates_the_phone_to_the_verified_number()
{
    var token = await ProveIdentityAsync();
    await RequestStep().ExecuteAsync(new RequestPhoneVerificationDto(Email, token,
NewPhone), "127.0.0.1", CancellationToken.None);

    await CheckStep().ExecuteAsync(new CheckPhoneVerificationDto(Email, NewPhone,
"123456"), CancellationToken.None);

    Assert.That(_sms.Checked.Last(), Is.EqualTo((NewPhone, "123456")));
    Assert.That(_identity.LastPhoneUpdate, Is.EqualTo((_profileId, NewPhone)));
}

[Test]
public async Task
Repeated_rejections_lock_the_challenge_even_if_twilio_would_keep_answering()
{
    var token = await ProveIdentityAsync();
    await RequestStep().ExecuteAsync(new RequestPhoneVerificationDto(Email, token,
NewPhone), "127.0.0.1", CancellationToken.None);
```

```
    _sms.NextResult = SmsCheckResult.Rejected;

    for (var i = 0; i < 4; i++)
    {
        Assert.ThrowsAsync<InvalidCodeException>(() =>
            CheckStep().ExecuteAsync(new CheckPhoneVerificationDto(Email, NewPhone,
"000000"), CancellationToken.None));
    }

    Assert.ThrowsAsync<TooManyAttemptsException>(() =>
        CheckStep().ExecuteAsync(new CheckPhoneVerificationDto(Email, NewPhone,
"000000"), CancellationToken.None));

    _sms.NextResult = SmsCheckResult.Approved;
    Assert.ThrowsAsync<TooManyAttemptsException>(() =>
        CheckStep().ExecuteAsync(new CheckPhoneVerificationDto(Email, NewPhone,
"123456"), CancellationToken.None));
    Assert.That(_identity.LastPhoneUpdate, Is.Null);
}
```

### VerificationCodeServiceTests.cs - Códigos de verificación:

```
[Test]
public async Task VerifyAsync_succeeds_with_the_correct_code()
{
    var (service, repo, email, _) = Build();
    var profileId = Guid.NewGuid();
    await service.IssueAsync(profileId, Purpose.PasswordReset, "user@example.com",
"127.0.0.1", CancellationToken.None);

    var code = ExtractCode(email.Sent[0].Body);
    var ticket = await service.VerifyAsync(profileId, Purpose.PasswordReset, code,
CancellationToken.None);

    Assert.That(ticket.ProfileId, Is.EqualTo(profileId));
    Assert.That(string.IsNullOrEmpty(ticket.ResetToken), Is.False);
}

[Test]
public async Task VerifyAsync_locks_after_reaching_MaxAttempts()
{
    var (service, _, email, _) = Build(new OtpOptions { ExpirationMinutes = 10,
MaxAttempts = 2, MaxRequestsPerWindow = 5, RequestWindowMinutes = 10,
ResetTokenExpirationMinutes = 5 });
    var profileId = Guid.NewGuid();
    await service.IssueAsync(profileId, Purpose.PasswordReset, "user@example.com",
"127.0.0.1", CancellationToken.None);

    Assert.ThrowsAsync<InvalidCodeException>(() => service.VerifyAsync(profileId,
Purpose.PasswordReset, "000000", CancellationToken.None));
    Assert.ThrowsAsync<TooManyAttemptsException>(() => service.VerifyAsync(profileId,
Purpose.PasswordReset, "000000", CancellationToken.None));

    var correctCode = ExtractCode(email.Sent[0].Body);
    Assert.ThrowsAsync<TooManyAttemptsException>(() => service.VerifyAsync(profileId,
Purpose.PasswordReset, correctCode, CancellationToken.None));
}
```

### SmtpEmailSenderTests.cs - Servidor SMTP fake in-process:

```
[Test]
public async Task Delivers_the_message_to_an_SMTP_server_without_credentials()
{
    using var server = new FakeSmtpServer();
    var sender = Build(new SmtpOptions { Host = "127.0.0.1", Port = server.Port,
EnableSsl = false, FromAddress = "no-reply@example.test", FromName = "Test",
TimeoutSeconds = 5 });

    await sender.SendAsync("anyone@example.org", "Verification code", "Your code is
123456", CancellationToken.None);

    var data = await server.Received.Task.WaitAsync(TimeSpan.FromSeconds(5));
    Assert.That(data, Does.Contain("anyone@example.org"));
    Assert.That(data, Does.Contain("charset=utf-8").IgnoreCase);

    var bodyStart = data.IndexOf("\r\n\r\n", StringComparison.Ordinal) + 4;
    var body =
Encoding.UTF8.GetString(Convert.FromBase64String(data[bodyStart..].Trim()));
    Assert.That(body, Does.Contain("Your code is 123456"));
}
```

### TransactionalEmailsTests.cs - Emails transaccionales:

```
[TestCase(Purpose.PasswordReset)]
[TestCase(Purpose.EmailChange)]
[TestCase(Purpose.EmailChangeConfirm)]
[TestCase(Purpose.PhoneChange)]
public void EveryPurposeRendersCodeInHtmlAndText(Purpose purpose)
{
    var email = TransactionalEmails.VerificationCode(Branding, purpose, "482917", 10);

    Assert.That(email.Html, Does.Contain("482917"));
    Assert.That(email.Text, Does.Contain("482917"));
    Assert.That(email.Text, Does.Contain("10 minutos"));
    Assert.That(email.Subject, Does.Contain("Guardian Escolar"));
    Assert.That(email.Html, Does.Not.Contain("<script").IgnoreCase);
    Assert.That(email.Html, Does.Contain("<CORREO_DE_SOPORTE>"));
}

[Test]
public void DynamicValuesAreHtmlEncoded()
{
    var email = TransactionalEmails.VerificationCode(Branding, Purpose.PasswordReset,
"1", 10, "<b>Eve</b>");

    Assert.That(email.Html, Does.Not.Contain("<b>Eve</b>"));
    Assert.That(email.Html, Does.Contain("<b>Eve</b>"));
}
```

## *Evidencia de Ejecución (Docker Compose)*

```
Test run for /src/tests/ms-forgot-information.Tests/bin/Debug/net10.0/ms-forgot-
information.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    62, Skipped:     0, Total:    62, Duration: 1 s
```

---

## Arquitectura de Pruebas y Patrones Utilizados

### Patrones Comunes en Todos los Servicios

1. **In-Memory Fakes / Test Doubles**
   - Repositorios en memoria (`InMemoryBusRepository`, `InMemoryFamilyRepository`, `FakeVerificationRequestRepository`)
   - Servicios externos simulados (`FakeIamService`, `FakeGpsDeviceService`, `FakeSmsVerificationService`, `FakeEmailSender`)
   - Proveedores de tiempo controlados (`FixedTimeProvider`, `ManualTimeProvider`)

2. **Arquitectura de Pruebas por Capas**
   - **Dominio**: Tests de modelos y reglas de negocio puras (`BusTests`, `AlertDescriptionBuilderTests`)
   - **Aplicación/Use Cases**: Tests de servicios de caso de uso con repositorios fake (`CreateBusServiceTests`, `ReportIncidentServiceTests`)
   - **Infraestructura**: Tests de adaptadores gRPC, controllers REST, interceptores (`RouteGrpcServiceTests`, `CampusControllerTests`, `InternalApiKeyInterceptorTest`)
   - **Integración**: Tests con base de datos en memoria (`ProcessScanServiceTests` con EF Core InMemory)

3. **Naming Convention**
   - `ExecuteAsync_<Escenario>_<ResultadoEsperado>` (C#)
   - `returnsXxx`, `rejectsXxx`, `issuesXxx` (Java)
   - Español para servicios .NET, inglés para Java (consistente con código base)

4. **Datos de Prueba**
   - Builders/Factory methods para crear entidades válidas (`NewBus`, `ValidRequest`, `ValidToken`)
   - Constantes para IDs fijos cuando se necesita determinismo (`SabanaId`, `CampusSabanaId`)

5. **Aislamiento**
   - Cada test usa nueva instancia de repositorios (no estado compartido)
   - `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]` en NUnit
   - Mocks frescos por test en Mockito

---

**Cobertura de Requerimientos Funcionales (FR)**

| FR | Descripción | Servicios | Tests Cubriendo |
|---|---|---|---|
| FR-2.1 | Autenticación y JWT | ms-iam | 6 (LoginServiceTest) |
| FR-2.2 | Renovación token acceso | ms-iam | 5 (RefreshTokenServiceTe |

| FR | Descripción | Servicios | Tests Cubriendo |
|---|---|---|---|
| | | | st) |
| FR-4.2 | Registro subida bus | ms-notification | 39 (ProcessScanServiceTests) |
| FR-4.3 | Registro bajada bus | ms-notification | Incluido en ProcessScanServiceTests |
| FR-5.1 | Detectar evento subida | ms-notification | Incluido en ProcessScanServiceTests |
| FR-8.1 | Conductor reporta incidente | ms-notification | 21 (ReportIncidentServiceTests) |
| FR-8.2 | Notificar escuela/acudientes | ms-notification, sg-ms-forgot-information | 21 + 10 (NotifyIncidentReportedServiceTests) |
| CU-FLEET-01 | Crear bus | ms-fleet | 3 |
| CU-FLEET-02 | Buscar buses | ms-fleet | 5 |
| CU-FLEET-03 | Asignar conductor | ms-fleet | 4 |
| CU-FLEET-04 | Bus asignado a conductor | ms-fleet | 4 |
| CU-FLEET-05 | Listar buses con detalles | ms-fleet | 8 |
| CU-FLEET-06 | Filtrado multi-tenant | ms-fleet | 5 |
| CU-FLEET-07 | gRPC interno ruta | ms-fleet | 7 |
| CU-ROUTE-01 | Crear ruta | ms-route | 3 |
| CU-ROUTE-02 | Asignar bus a ruta | ms-route | 4 |
| CU-ROUTE-03 | Ruta conductor hoy | ms-route | 25 |
| CU-SCHOOL-01 | Obtener escuela | ms-school-management | 3 |
| CU-SCHOOL-02 | Buscar escuelas | ms-school-management | 4 |
| CU-SCHOOL-03 | Listar escuelas tenant | ms-school-management | 5 |
| CU-SCHOOL-04 | Sede central creación/update | ms-school-management | 8 |
| CU-USER-01 | Registrar familia | ms-user-management | 5 |
| CU-USER-02 | Licencia conductor | ms-user-management | 8 |
| CU-USER-03 | Buscar personas | ms-user-management | 5 |
| CU-FORGOT-01 | Olvidé contraseña | sg-ms-forgot-information | 2 |
| CU-FORGOT-02 | Cambio teléfono | sg-ms-forgot-information | 31 |
| CU-FORGOT-03 | Códigos verificación | sg-ms-forgot-information | 10 |

# Métricas de Calidad

## Resumen Global

| Métrica | Valor |
|---|---|
| **Total Tests** | 376 |
| **Tests Pasados** | 376 (100%) |
| **Tests Fallados** | 0 (0%) |
| **Tests Omitidos** | 0 (0%) |
| **Tiempo Total Ejecución** | ~12 segundos (paralelo por servicio) |
| **Servicios con Tests** | 7/7 (excluyendo GPS) |

## Por Servicio

| Servicio | Tests | Tiempo | Cobertura Estimada* |
|---|---|---|---|
| ms-fleet | 39 | 313 ms | ~85% (Use Cases + Domain) |
| ms-iam | 57 | 72 s | ~90% (Use Cases + Adapters + Config) |
| ms-notification | 56 | 3 s | ~88% (Use Cases + Domain + Infrastructure) |
| ms-route | 74 | 1 s | ~92% (Use Cases completos + Edge cases temporales) |
| ms-school-management | 22 | 817 ms | ~80% (Use Cases + Controller) |
| ms-user-management | 66 | 1 s | ~87% (Use Cases + Domain + Family + Driver) |
| sg-ms-forgot-information | 62 | 1 s | ~90% (Use Cases + Infrastructure + Email) |

*Estimación basada en casos de uso cubiertos vs. totales en código.

# Conclusiones

## ✅ Fortalezas

1. **Cobertura Completa**: Todos los 7 servicios evaluados tienen suites de pruebas unitarias robustas que cubren casos de uso principales, edge cases, validaciones y reglas de negocio.
2. **Aislamiento Correcto**: Uso extensivo de fakes/in-memory repositories evita dependencias externas (BD, Kafka, SMTP, gRPC), haciendo tests rápidos y deterministas.
3. **Patrones Consistentes**: Cada servicio sigue patrones de testing apropiados a su stack tecnológico (NUnit + Fakes para .NET, JUnit 5 + Mockito para Java).
4. **Tests de Resiliencia**: Pruebas que verifican comportamiento ante fallos de dependencias externas (directorios caídos, Twilio errors, SMTP failures).
5. **Validación de Reglas de Negocio**: Tests exhaustivos de invariantes de dominio (placas únicas, conductor único por bus, idempotencia escaneos, rate limiting OTP, filtrado multi-tenant).
6. **Ejecución Automatizada**: Todos los servicios incluyen perfil `tests` en docker-compose.yml para ejecución reproducente en CI/CD.

## 📋 Recomendaciones

1. **Cobertura de Integración**: Considerar agregar tests de integración con Testcontainers para validar interacciones reales con PostgreSQL, Kafka.
2. **Tests de Contrato gRPC**: Para servicios que exponen gRPC (ms-iam, ms-fleet, ms-route, ms-school-management), agregar tests de compatibilidad de contratos protobuf.
3. **Mutation Testing**: Evaluar introducción de Stryker.NET / Pitest para medir calidad real de aserciones.
4. **Documentación de Casos de Prueba**: Mantener este informe actualizado como documentación viva del comportamiento esperado.
5. **Pipeline CI/CD**: Integrar ejecución de `docker compose --profile tests run --rm <service>.tests` en pipeline de PR/merge.

---

## Anexos

### A. Comandos de Ejecución por Servicio

```
# ms-fleet
cd backend/ms-fleet && docker compose --profile tests run --rm ms-fleet.tests

# ms-iam
cd backend/ms-iam && docker run --rm -v "$(pwd)":/src -w /src maven:3.9.6-eclipse-
temurin-21 mvn test

# ms-notification
cd backend/ms-notification && docker compose --profile tests run --rm ms-
notification.tests

# ms-route
cd backend/ms-route && docker compose --profile tests run --rm ms-route.tests

# ms-school-management
cd backend/ms-school-management && docker compose --profile tests run --rm ms-school-
management.tests
```

```
# ms-user-management
cd backend/ms-user-management && docker compose --profile tests run --rm ms-user-
management.tests

# sg-ms-forgot-information
cd backend/sg-ms-forgot-information && docker compose --profile tests run --rm ms-
forgot-information.tests
```

**B. Requisitos Previos**

- Docker y Docker Compose instalados
- Red `sg-services-network` creada (`docker network create sg-services-network`)
- Imágenes base disponibles (mcr.microsoft.com/dotnet/sdk:10.0, maven:3.9.6-eclipse-temurin-21)

**C. Archivos de Configuración de Tests**

Cada servicio incluye su `docker-compose.yml` con perfil `tests` que:

- Usa imagen SDK de .NET 10 / Maven para Java
- Monta el código fuente como volumen
- Ejecuta `dotnet test` / `mvn test` en directorio de tests
- Conecta a red `sg-services-network` (aunque tests unitarios no la requieran)

---

**Fin del Informe**

*Documento generado automáticamente tras ejecución de suites de pruebas unitarias en contenedores Docker Compose.*